Day 143 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 21, Day 4 reference

Watch: OpenAI's CLIP Explained and Implementation | Contrastive Learning (Karndeep Singh) | CLIP: Learning Transferable Visual Models From Natural Language Supervision (KnowledgeDB AI)

`day143_clip_joint_embeddings.py`, segment by segment, in the order `main()` uses things. The measurements, the caption writer, the bag of words, the contrastive loss and its gradient, the training loop, both embeddings, zero-shot naming and retrieval are new; the networks and Adam are Day 126's, imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: u, v = image and caption vectors scaled to length 1 · S = similarity table · τ = TAU, the temperature · n = batch size

Segment 0 — Setup

Imports and constants: a vocabulary and a word index

CODE

```
NUM = "zero one two three four five six seven eight nine".split()
VOCAB = NUM + "a the digit handwritten written thin thick leaning left right upright and".split()
IDX = {w: i for i, w in enumerate(VOCAB)}
ACTS = ["relu", "linear"]
TAU = 0.1
EPOCHS = 10
HOLD = (7, 1)
```

- `"zero one ... nine".split()` — a string split on spaces: a quick way to write a list of words.
- `NUM + "a the digit ...".split()` — + joins two lists: the digit words come first, so word k < 10 is digit k.
- `{w: i for i, w in enumerate(VOCAB)}` — a dictionary comprehension: each word maps to its position, its column in the bag of words.
- `HOLD = (7, 1)` — a (digit, thickness) pair: seven, thick.

Segment 1 — Captions

measure: two attributes for 70,000 digits at once

MATH

$$lean = -\Sigma\, p\,(r - r_0)(c - c_0) \,/\, \Sigma\, p\,(r - r_0)^2 \text{ (a weighted regression of column on row)}$$

CODE

```
def measure(X, y, n_train=60000):
    ink = X.mean(1)
    R = np.repeat(np.arange(28.0), 28)[None]; Cc = np.tile(np.arange(28.0), 28)[None]
    m = X.sum(1, keepdims=True) + 1e-9; mr = (X * R).sum(1, keepdims=True) / m; mc = (X *
        Cc).sum(1, keepdims=True) / m
    slant = -(X * (R - mr) * (Cc - mc)).sum(1) / (X * (R - mr) ** 2).sum(1)
    thick = np.zeros(len(X), int); lean = np.zeros(len(X), int)
    for d in range(10):
        tr = y[:n_train] == d; k = y == d
        a1, a2 = np.percentile(ink[:n_train][tr], [100 / 3, 200 / 3]); b1, b2 =
            np.percentile(slant[:n_train][tr], [100 / 3, 200 / 3])
        thick[k] = np.where(ink[k] < a1, -1, np.where(ink[k] > a2, 1, 0))
        lean[k] = np.where(slant[k] < b1, -1, np.where(slant[k] > b2, 1, 0))
    return ink, slant, thick, lean
```

- `np.repeat(np.arange(28.0), 28)[None]` — 0,0,...,0,1,1,...: each pixel's row; [None] adds a first axis so it lines up with every image.
- `np.tile(np.arange(28.0), 28)[None]` — 0,1,...,27,0,1,...: each pixel's column.
- `np.percentile(ink[:n_train][tr], [100 / 3, 200 / 3])` — the two cut points that split one digit's training values into thirds.
- `np.where(ink[k] < a1, -1, np.where(ink[k] > a2, 1, 0))` — a nested choice for every element at once: –1, 1 or 0.

caption and bow

CODE

```
def caption(d, th, ln, rng, hold=None):
    w = [NUM[d]]
    if th != 0 and rng.rand() < 0.6 and (d, th) != hold:
        w.append("thick" if th > 0 else "thin")
    if ln != 0 and rng.rand() < 0.6:
        w += ["leaning", "right" if ln > 0 else "left"]
    elif ln == 0 and rng.rand() < 0.3:
        w.append("upright")
    return w + [["a"], ["the", "digit"], ["handwritten"], ["written"], []][rng.randint(5)]
def bow(captions):
    B = np.zeros((len(captions), len(VOCAB)), np.float32)
    for r, ws in enumerate(captions):
        for w in ws:
            if w in IDX:
                B[r, IDX[w]] += 1
        B[r] /= max(B[r].sum(), 1)
    return B
```

- `(d, th) != hold` — tuples compare item by item: True unless this is the held-out pair (always True when hold is None).
- `w += ["leaning", ...]` — += on a list adds every item of the right-hand list.
- `[["a"], ["the", "digit"], ...][rng.randint(5)]` — a list of five word lists, indexed by a random number: one filler chosen at random.

- `if w in IDX:` — unknown words are skipped: the reason section 4's unknown words change nothing.
- `B[r] /= max(B[r].sum(), 1)` — divide by the word count; max(..., 1) avoids dividing by zero for a caption with no known words.

Segment 2 — The loss and training

clip_loss: both softmaxes and the gradient

MATH

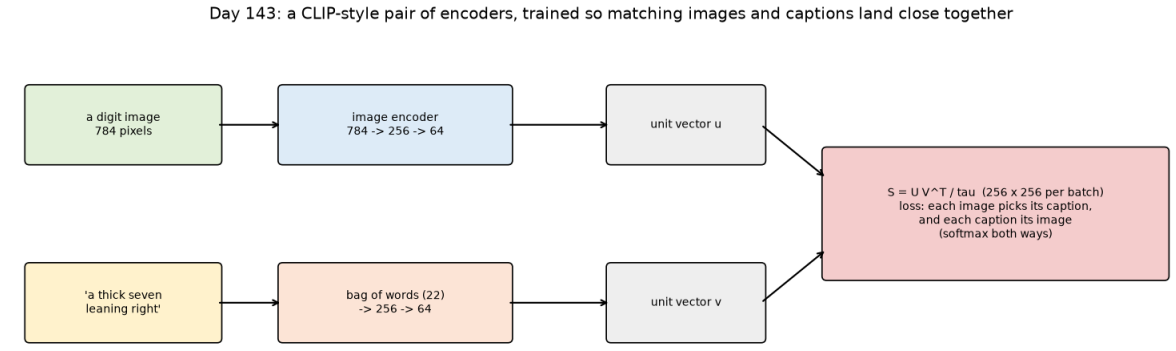
$$dS = (P_{row} - I + P_{col} - I) / 2n; \quad dU = dS\, V / \tau; \quad dV = dS^T\, U / \tau; \quad dz = (du - u\,(u \cdot du)) / |z|$$

CODE

```
def clip_loss(Zi, Zt, tau=TAU):
    ni = np.linalg.norm(Zi, axis=1, keepdims=True) + 1e-12; nt = np.linalg.norm(Zt, axis=1,
        keepdims=True) + 1e-12
    U, V = Zi / ni, Zt / nt; S = U @ V.T / tau; n = len(S); I = np.arange(n)
    P1 = np.exp(S - S.max(1, keepdims=True)); P1 /= P1.sum(1, keepdims=True)
    P2 = np.exp(S - S.max(0, keepdims=True)); P2 /= P2.sum(0, keepdims=True)
    loss = float(-(np.log(P1[I, I] + 1e-30).mean() + np.log(P2[I, I] + 1e-30).mean()) / 2)
    G1 = P1.copy(); G1[I, I] -= 1; G2 = P2.copy(); G2[I, I] -= 1; dS = (G1 + G2) / (2 * n)
    dU = dS @ V / tau; dV = dS.T @ U / tau
    return loss, (dU - U * (U * dU).sum(1, keepdims=True)) / ni, (dV - V * (V * dV).sum(1,
        keepdims=True)) / nt
```

- `S.max(1, keepdims=True)` — subtracting each row's largest value before exp keeps the numbers from overflowing; the softmax is unchanged.
- `P2 = np.exp(S - S.max(0, keepdims=True))` — the same along columns (axis 0): each caption's softmax over the images.
- `P1[I, I]` — two index arrays: elements (0,0), (1,1), ...: the diagonal, the matching pairs.
- `G1[I, I] -= 1` — softmax with cross-entropy: the gradient is the probabilities minus 1 at the right answer.
- `dU - U * (U * dU).sum(1, keepdims=True)` — removes from each row's gradient its part along u itself; divided by |z| it is the gradient before scaling.

SYSTEM DIAGRAM



The two encoders and the loss.

train_clip: two networks, one loss

CODE

```
def train_clip(X, y, thick, lean, epochs=EPOCHS, hold=None, seed=0, batch=256, lr=1e-3,
    every=None):
    img = d126.init_net([784, 256, 64], seed); txt = d126.init_net([len(VOCAB), 256, 64], seed +
        1)
    oi, ot = d126.Adam(img, lr), d126.Adam(txt, lr); rng = np.random.RandomState(seed); log = []
    for ep in range(1, epochs + 1):
        o = rng.permutation(len(X)); L = []
        for s in range(0, len(X) - batch + 1, batch):
            b = o[s:s + batch]; caps = [caption(y[i], thick[i], lean[i], rng, hold) for i in b]
            Ai = d126.forward(img, X[b], ACTS); At = d126.forward(txt, bow(caps), ACTS)
            loss, gi, gt = clip_loss(Ai[-1].astype(np.float64), At[-1].astype(np.float64))
            oi.step(img, d126.backward(img, Ai, ACTS, gi.astype(np.float32))[0])
            ot.step(txt, d126.backward(txt, At, ACTS, gt.astype(np.float32))[0]); L.append(loss)
        log.append((ep, float(np.mean(L)), every(img, txt) if every else None))
    return img, txt, log
```

- `init_net([784, 256, 64], seed); init_net([len(VOCAB), 256, 64], seed + 1)` — two networks with different seeds, so they do not start identical.
- `range(0, len(X) - batch + 1, batch)` — only full batches: the last 96 digits of each shuffle are skipped.
- `Ai[-1].astype(np.float64)` — the loss is computed in double precision; the gradient goes back as float32, the networks' type.
- `every(img, txt) if every else None` — an optional function called once per epoch: here, zero-shot accuracy.

embed_images, embed_texts, zero_shot

CODE

```
def embed_images(img, X):
    Z = d126.forward(img, X, ACTS)[-1].astype(np.float64); return Z / np.linalg.norm(Z, axis=1,
        keepdims=True)
def embed_texts(txt, captions):
    Z = d126.forward(txt, bow(captions), ACTS)[-1].astype(np.float64); return Z /
        np.linalg.norm(Z, axis=1, keepdims=True)
def zero_shot(img, txt, X, y, templates=("the digit {}",)):
    T = sum(embed_texts(txt, [t.format(w).split() for w in NUM]) for t in templates)
    T /= np.linalg.norm(T, axis=1, keepdims=True)
    return float(((embed_images(img, X) @ T.T).argmax(1) == y).mean())
```

- `Z / np.linalg.norm(Z, axis=1, keepdims=True)` — every row scaled to length 1, so a dot product is a cosine.
- `[t.format(w).split() for w in NUM]` — 'the digit {}'.format('seven') gives 'the digit seven', split into words: ten prompts.
- `sum(embed_texts(...) for t in templates)` — sum over a generator adds the ten-row arrays: one summed vector per digit.
- `(E @ T.T).argmax(1)` — a 2,000 × 10 table of similarities; argmax picks each image's closest prompt.

Segment 3 — The experiments

demo_hand: the gradient check, every entry

CODE

```
rng = np.random.RandomState(0); A = rng.randn(6, 4); B = rng.randn(6, 4); _, gA, gB = clip_loss(A,
    B); worst = 0.0
for M, G in ((A, gA), (B, gB)):
    for idx in np.ndindex(M.shape):
        old = M[idx]; M[idx] = old + 1e-6; up = clip_loss(A, B)[0]; M[idx] = old - 1e-6; dn =
            clip_loss(A, B)[0]; M[idx] = old
        worst = max(worst, abs((up - dn) / 2e-6 - G[idx]))
```

- `for idx in np.ndindex(M.shape):` — every (row, column) index of the array in turn: 24 per array.
- `M[idx] = old + 1e-6; ...; M[idx] = old` — M is A or B itself, so changing it changes the loss; the value is put back each time.

demo_zero_shot: what the prompt changes

CODE

```
for t in ("{}", "a {}", "the digit {}", "handwritten {}", "a photo of the number {}", "a thick
    {}", "a thin {} leaning left",
          "{} upright and thick", "not a {}"):
    unk = [w for w in t.replace("{}", "").split() if w not in IDX]
    print(f"{repr(t):>36} {zero_shot(img, txt, X[te], y[te], (t,)):9.4f}   {' '.join(unk) if unk
        else '-'}")
ens = ("{}", "a {}", "the digit {}", "handwritten {}", "written {}")
print(f"{'ensemble of 5 plain prompts':>36} {zero_shot(img, txt, X[te], y[te], ens):9.4f}")
a = embed_texts(txt, ["a thin seven leaning left".split()]); b = embed_texts(txt, ["left seven
    thin leaning a".split()])
c = embed_texts(txt, ["a seven".split()]); d = embed_texts(txt, ["a seven".split() + ["purple",
    "elephant"]])
print(f"word order: 'a thin seven leaning left' against 'left seven thin leaning a': cosine
    {float((a @ b.T)[0, 0]):.6f}")
print(f"unknown words: 'a seven' against 'a seven purple elephant': cosine {float((c @ d.T)[0,
    0]):.6f}")
```

- `[w for w in t.replace("{}", "").split() if w not in IDX]` — the template's words, without the slot, that the vocabulary lacks.
- `repr(t)` — the string with its quotes, so the bare template '{}' is visible.
- `float((a @ b.T)[0, 0])` — a (1, 64) by (64, 1) product is a 1 × 1 array; [0, 0] takes its one number. The first draft passed the array to float() and stopped with a TypeError.

retrieve and demo_retrieval: whole words, not substrings

CODE

```
def retrieve(img, txt, X, words, pool, k=50):
    return pool[np.argsort(-(embed_images(img, X[pool]) @ embed_texts(txt,
        [words.split()])[0]))[:k]]
```

- `np.argsort(-(...))[:k]` — argsort sorts ascending, so the similarities are negated: the k most similar first.
- `pool[...]` — the sorted positions index back into the pool to give image numbers.

CODE

```
top = retrieve(img, txt, X, q, pool); ws = q.split(); d = next(NUM.index(w) for w in ws if w in
    NUM)
att = thick if ("thick" in ws or "thin" in ws) else lean; want = 1 if ("thick" in ws or "right" in
    ws) else (-1 if ("thin" in ws or "left" in ws) else 0)
```

- `"right" in ws` — ws is a list of words, so this asks whether 'right' is one of them. The first draft tested `"right" in q` on the string, which is True for 'upright': 'a thin eight upright' was checked as thick.

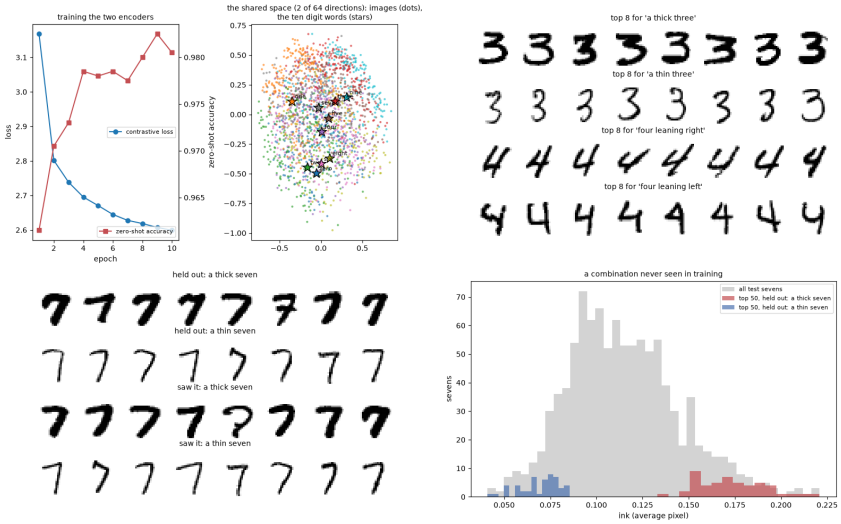
demo_held_out: a second pair with one combination withheld

CODE

```
i2, t2, _ = train_clip(X[:60000], y[:60000], thick[:60000], lean[:60000], hold=HOLD)
n_hold = sum(1 for i in range(60000) if y[i] == 7 and thick[i] == 1)
print(f"(training sevens measured thick, never called thick: {n_hold}; zero-shot accuracy of the
    held-out encoders "
      f"{zero_shot(i2, t2, X[te], y[te]):.4f})")
print(f"{'encoders':>14} {'query':>16} {'thick share, top 50':>20} {'mean ink, top 50':>17}")
for name, a, b in (("saw it", img, txt), ("held out", i2, t2)):
    for q in ("a thick seven", "a thin seven", "a thick three"):
        pool = sev if "seven" in q else np.arange(60000, 70000)[y[60000:] == 3]; top = retrieve(a,
            b, X, q, pool)
        print(f"{name:>14} {q:>16} {np.mean(thick[top] == (1 if 'thick' in q else -1)):20.2f}
            {ink[top].mean():17.4f}")
```

- `hold=HOLD` — the only change from section 3: caption() never says 'thick' for a thick seven.
- `sum(1 for i in range(60000) if ...)` — counts the matches by adding 1 for each.
- `for name, a, b in (("saw it", img, txt), ("held out", i2, t2)):` — both pairs of encoders run through the same queries.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `"a b c".split()` — a list of words.
- `{k: v for ...}` — a dictionary comprehension.
- `np.ndindex(shape)` — every index of an array of that shape.
- `A[I, I]` — the diagonal, by two index arrays.
- `"right" in ws vs "right" in q` — membership in a list of words vs a substring of a string.
- `float(a[0, 0])` — one number from a 1 × 1 array.

End of Day 143 syntax reference. For the experiments and what they showed, see the Day 143 complete guide.